HPC-GPT: Integrating Large Language Model for High-Performance Computing

Xianzhong Ding
xding5@ucmerced.edu
University of California, Merced
Merced, CA, USA
Argonne National Laboratory
Lemont, IL, USA

Le Chen
lechen@iastate.edu
Iowa State University
Ames, IA, USA
Lawrence Livermore National Laboratory
Livermore, CA, USA

Murali Emani
memani@anl.gov
Argonne National Laboratory
Lemont, IL, USA

Chunhua Liao
liao6@llnl.gov
Lawrence Livermore National Laboratory
Livermore, CA, USA

Pei-Hung Lin
lin32@llnl.gov
Lawrence Livermore National Laboratory
Livermore, CA, USA

Tristan Vanderbruggen
vanderbrugge1@llnl.gov
Lawrence Livermore National Laboratory
Livermore, CA, USA

Zhen Xie
zxie3@binghamton.edu
Binghamton University
Binghamton, NY, USA

Alberto E. Cerpa
acerpa@ucmerced.edu
University of California, Merced
Merced, CA, USA

Wan Du
wdu3@ucmerced.edu
University of California, Merced
Merced, CA, USA

ABSTRACT

Large Language Models (LLMs), including the LLaMA model, have exhibited their efficacy across various general-domain natural language processing (NLP) tasks. However, their performance in high-performance computing (HPC) domain tasks has been less than optimal due to the specialized expertise required to interpret the model's responses. In response to this challenge, we propose HPC-GPT, a novel LLaMA-based model that has been supervised fine-tuning using generated QA (Question-Answer) instances for the HPC domain. To evaluate its effectiveness, we concentrate on two HPC tasks: managing AI models and datasets for HPC, and data race detection. By employing HPC-GPT, we demonstrate comparable performance with existing methods on both tasks, exemplifying its excellence in HPC-related scenarios. Our experiments on open-source benchmarks yield extensive results, underscoring HPC-GPT's potential to bridge the performance gap between LLMs and HPC-specific tasks. With HPC-GPT, we aim to pave the way for LLMs to excel in HPC domains, simplifying the utilization of language models in complex computing applications.

CCS CONCEPTS

• **Software and its engineering**; • **Computing methodologies** → **Artificial intelligence**; **Parallel algorithms**;



This work is licensed under a Creative Commons Attribution International 4.0 License.

SC-W 2023, November 12–17, 2023, Denver, CO, USA
© 2023 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0785-8/23/11.
https://doi.org/10.1145/3624062.3624172

KEYWORDS

High-performance Computing, Data Race Detection, Large Language Model, OpenMP, Neural Network.

ACM Reference Format:
Xianzhong Ding, Le Chen, Murali Emani, Chunhua Liao, Pei-Hung Lin, Tristan Vanderbruggen, Zhen Xie, Alberto E. Cerpa, and Wan Du. 2023. HPC-GPT: Integrating Large Language Model for High-Performance Computing. In *Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis (SC-W 2023), November 12–17, 2023, Denver, CO, USA.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3624062.3624172

1 INTRODUCTION

Deep learning has found application across various domains and has been utilized in a multitude of applications, including code generation [30, 32], building control [21, 22], irrigation scheduling [19, 20], and even in tasks related to epistemic uncertainty and occupancy estimation [6, 41]. These applications showcase the versatility and adaptability of deep learning techniques. On the other hand, language models (LMs), particularly large language models (LLMs) trained on extensive textual data, have recently demonstrated remarkable capabilities in various natural language processing and visualization tasks, reflecting their growing importance in the field of artificial intelligence. They have also been widely used to process programming languages due to the similarities between natural languages and programming languages. Based on an LLM trained on code [16], GitHub provides an AI assistant for developing software.

With the surging popularity of LLMs, the high-performance computing (HPC) community is exploring their potential to tackle various HPC challenges like programming language processing, parallel programming, and question answering. However, existing pre-trained LLMs were initially designed for general tasks, such as

dialogue and article summarization, which limits their effectiveness in HPC applications due to a lack of relevant domain knowledge. Several attempts have been made to employ LLMs for HPC tasks. For instance, Godoy et al. [24] evaluated the capacity of OpenAI Codex [4] via Copilot for generating HPC numerical kernels, while Chen et al. [12] developed pipelines to support common HPC tasks like code similarity analysis and parallelism detection. However, they are still in the process of investigating the adequacy of existing LLMs for HPC tasks. The general LLMs perform poorly on specific HPC tasks. For example, in question-answering tasks related to HPC, such as OpenMP Q&A [12], even the latest ChatGPT fails to provide accurate answers. This is because general LLMs lack the specific knowledge required to solve HPC-related challenges, despite being trained on large datasets that may include HPC-related information. Although the generalization ability of LLMs improves with more diverse training data from various domains, their performance in a specific domain, like HPC, reduces.

In this paper, we aim to explore and train an open-source LLM tailored for HPC applications. Our focus revolves around two general applications, each comprising two subtasks. The first application focuses on managing AI models and datasets for programming language processing (PLP) [23] and MLPerf [42] tasks. Through datasets and model information collection, we aim to help users better comprehend the required computing resources for training AI models and datasets. Liao et al. [34] proposed HPC ontology to capture HPC-related concepts using the web ontology language (OWL) [39]. HPC ontology allows users to query PLP and HPC information but is limited by the manual processes of data creation, collection, and updates. Moreover, HPC ontology depends on users to articulate their requirements, including conditions or constraints related to code, hardware, dataset, models, and more, resulting in a challenge to accurately capture and utilize such nuanced semantics.

In the second application, we evaluate data race detection for C/C++ and Fortran tasks using the public data race benchmark (DRB) [35]. Data race detection analyses can be broadly categorized into dynamic and static approaches. While dynamic approaches, like Intel Inspector [29], are commonly used, there is growing interest in static data race detection tools that complement dynamic approaches. LLOV [8] is a recent tool evaluated with DRB. However, designing such tools demands significant data race expertise and running code snippets, making the process time-consuming.

Leveraging LLMs for solving HPC tasks holds tremendous promise, as a single HPC-domain LLM can effectively handle various general applications, eliminating the need to devise separate methods for each task. However, training an LLM for the HPC domain [36, 37] presents non-trivial challenges, primarily revolving around the collection of domain-specific training data. In contrast to traditional language model training, which involves feeding large volumes of raw data like text, fine-tuning an LLM for HPC necessitates specialized instruction data (e.g., PromptInstructions [7] and Super-Natural-Instructions [49]). These instructions guide the model's understanding of HPC concepts and tasks, ensuring it performs well in this domain. However, acquiring such instruction data proves to be a daunting task due to the associated costs and limitations in diversity. To address this issue, Wang et al. [46] propose a semi-automated process for instruction-tuning a pre-trained language model using instructional signals extracted from the model itself. This approach mitigates some of the challenges in data collection and provides an alternative means of training the LLM for HPC tasks. However, it requires a set of manually-written tasks to guide the overall generation process, which can be time-consuming and resource-intensive.

In this paper, we present HPC-GPT, a specialized LLM tailored for the HPC domain. To facilitate fine-tuning, we design an automatic collection method to gather instruction data for training the open-source base LLaMa [44] and LLaMa2 model [45]. Leveraging the power of the existing LLM (GPT-4), we generate domain-specific and contextually relevant instructions from unsupervised text. This approach ensures that the generated instructions are specific to the HPC domain and aligned with the provided unsupervised text. To address specific HPC tasks, we design distinct prompts that outline relevant requirements, enabling the model to generate instructions and answers separately. Subsequently, we establish filtering and pruning rules to eliminate instruction data that fails to meet the specified requirements. Our HPC-GPT model is built upon the foundation of the open-source LLaMa-13B base model, further enhancing its language generation capabilities. To showcase the potential of HPC-GPT, we utilize the two HPC applications mentioned earlier as illustrative examples to evaluate the performance. We summarize the main contributions of this paper as follows:

- Introduction of the HPC-GPT model, representing the first open-source LLM fine-tuned using HPC instruction data. This specialized model is specifically designed to excel in HPC tasks.
- Integration of HPC knowledge into the model, ensuring it possesses accurate and domain-specific information. By incorporating HPC knowledge, the model leads to enhanced performance in HPC applications.
- Release of two open-source datasets for the HPC domain: one containing HPC models and datasets, and the other dedicated to data race detection. These datasets provide valuable resources for researchers in the HPC community, facilitating further advancements in the field. The code [1] and datasets [2] are publicly available.
- Comparative evaluation against state-of-the-art methods. This progress underscores the model's effectiveness and its potential to outperform existing approaches in HPC-related tasks.

2 RELATED WORK

2.1 Large Language Models

In the realm of language processing, recent strides in LLMs [3, 10, 17, 18] have showcased their superiority over earlier paradigms like pretraining [31] and fine-tuning [26]. This remarkable progress can be attributed to the substantial growth in model scale, resulting in qualitative shifts within LLMs, known as emergent abilities. These newfound capabilities encompass in-context learning, enabling the model to tackle zero-shot tasks, and the ability to follow chains of thought [48], thereby enhancing its performance on intricate tasks.

OpenAI's groundbreaking work in developing ChatGPT [3] and GPT-4 [10] has brought about a paradigm shift in the understanding

[1] https://github.com/dingxianzhong/HPC-GPT
[2] https://huggingface.co/datasets/HPC-GPT/HPC

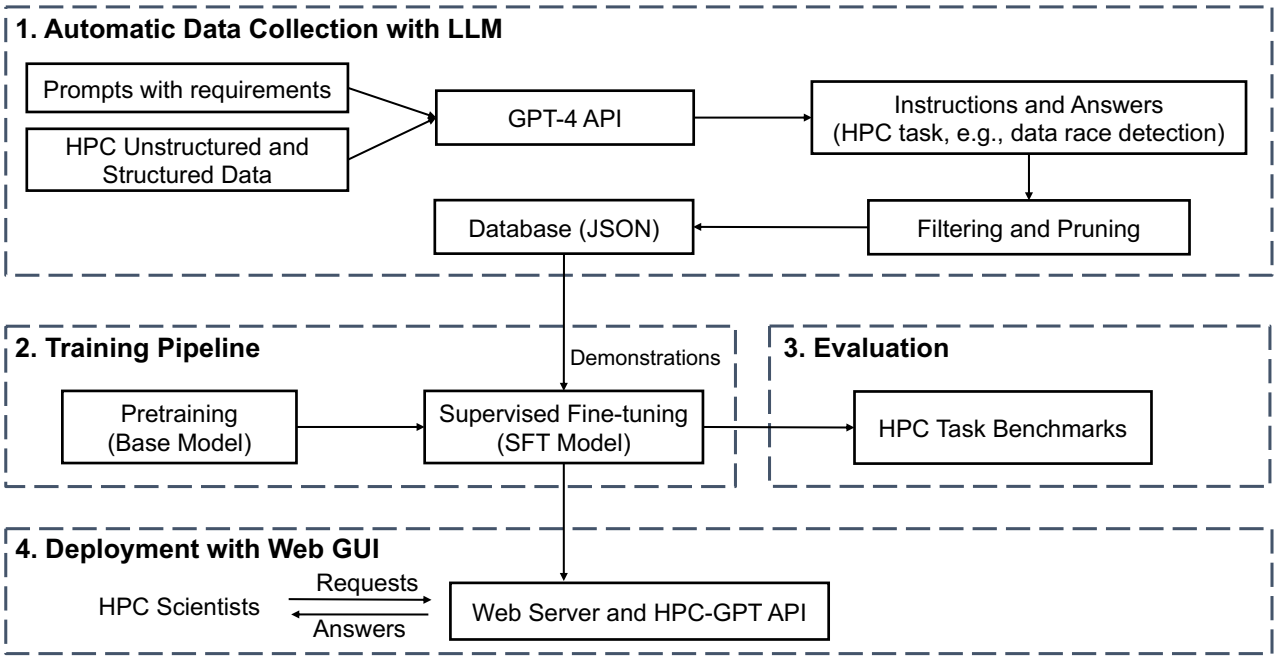


Figure 1: HPC-GPT Architecture.

of LLMs. These models have demonstrated remarkable performance, yet OpenAI has maintained confidentiality regarding their training strategies and weight parameters. To address this limitation, LLaMa [44] and LLaMa2 [45] emerge as an open-source alternative to GPT, available in sizes ranging from 7 billion to 65 billion parameters. While LLaMa's performance is comparable to GPT-3.5 in general tasks, it falls short in HPC-related tasks due to its training data being predominantly focused on general applications.

2.2 Large Language Models for HPC

LLMs, such as GPT-4 and LLaMA, have been widely used in multiple domains, including natural language processing, visualization, and so on. However, applying them for analyzing and optimizing HPC tasks is still challenging due to the lack of HPC-specific support. To address this challenge, LM4HPC [12] represents the first attempt to adapt LLMs to the HPC domain. This is achieved by creating the LM4HPC framework to facilitate research into HPC analyses and optimizations using LMs. Tailored for supporting HPC datasets, AI models, and pipelines, the LM4HPC framework is built on top of a range of components from different levels of the machine learning software stack, with Hugging Face-compatible APIs. However, the framework currently is still relying on existing general LLM for HPC tasks which is suboptimal.

3 DESIGN OF HPC-GPT

In this section, we describe in detail the design of HPC-GPT.

3.1 HPC-GPT Overview

Figure 1 illustrates an overview of the proposed HPC-GPTframework, which can be divided into four main stages: HPC domain data collection, training, evaluation, and deployment. In the HPC domain data collection stage, we develop a data collection method that automatically gathers the necessary training data for two specific HPC applications. This data curation process ensures that the model is trained on relevant and domain-specific information. Moving on to the training stage, we employ supervised fine-tuning on the open-source LLM using the generated instruction data. Fine-tuning helps the model adapt to the intricacies of HPC-related tasks and enhances its performance in the targeted domain. In the evaluation stage, we rigorously assess the well-trained HPC-GPT model's capabilities on a public data race detection dataset. This evaluation process validates the model's effectiveness and performance in real-world scenarios. Finally, after successful training and evaluation, we deploy the HPC-GPT model to a web server, making it readily available for HPC scientists and researchers to utilize in their work.

3.2 Automatic Data Collection with LLM

Instruction Generation: In this stage, we leverage the capabilities of the existing LLM, GPT-4, to generate domain-specific instructions based on unsupervised text, encompassing both unstructured and structured HPC data. The unsupervised text serves as the input and includes a wide range of HPC-related knowledge data. To tailor the instructions for specific HPC tasks, we design instruction prompts with relevant requirements, as depicted in Listing 1. These prompts guide GPT-4 to respond with instructions that align with the provided text data. The generated instructions serve as guidance for the subsequent stages of the process.

Task	Dataset Name	Language
Defect Detection	Devign	c
Code Repair	Bugs2Fix	Java

⇩

A task called "Defect Detection" along with the corresponding dataset name and programming language used. The dataset used for this task is called "Devign," and the programming language employed is C.

Figure 2: Transformation of unsupervised structured data.

```
"The HPC knowledge is:

{unsupervised knowledge data}

According to the information above, please help me
    generate {number} questions.

Here are the requirements:
1. Try not to repeat the verb for each question to
    maximize diversity.
2. Make sure the output is less than 50 words.
3. The questions can be asked under many conditions.
4. Do not generate the same or similar questions as
    generated before.

Now, please generate the instructions following the above
     requirements."
```

Listing 1: Instruction Generation Prompt

The **unsupervised knowledge data** within the prompt consists of sequential text. Unstructured knowledge data, such as websites and conference papers, can be directly used after undergoing necessary cleaning processes. On the other hand, structured data, like tables, needs to be converted into unstructured textual data before it becomes usable. As illustrated in Figure 2, this conversion can be achieved through slot-filling using templates or by concatenating each data entry with its corresponding attribute name. These steps ensure that structured data is transformed into a format that can be effectively utilized by the language model for instruction generation and fine-tuning.

Answer Generation: In this stage, the language model is tasked with generating answers to the instruction questions based on the corresponding unsupervised HPC knowledge. The process can be expressed as $A = P(K, Q)$, where K, Q, and A represent unsupervised knowledge, instruction question, and answer, respectively. Similar to the previous stage, the prompt for generating answers is provided, as shown in Listing 2. Using the instruction questions and the associated unsupervised knowledge, the language model is prompted to produce accurate and contextually relevant answers. This step enhances the model's understanding of the provided knowledge and strengthens its ability to respond effectively to the instruction questions.

Filtering and Pruning: Although we explicitly instruct the model with the prompt in Listing 2 to not generate the same or similar instructions and answers as generated before, we still observe that the model produces instruction data that violates these rules. Additionally, the generated instances of instructions also exhibit cases where they do not adhere to the required format and become unparseable. Therefore, it is necessary to further filter out these problematic examples. To mitigate these issues, we implement a postprocessing step to filter out inappropriate responses and correct any formatting errors. This involves developing heuristics and rule-based methods to identify and remove instances that violate the instructed constraints. By applying these filters, we ensure that the generated text adheres to the predefined guidelines and maintains the desired level of correctness.

```
"The HPC knowledge is:

{unsupervised knowledge data}.

Please answer the following question based on the above
    knowledge:
{the generated instruction}

Here are the requirements:
1. Try not to repeat the verb for each answer to maximize
     diversity.
2. Make sure the output is less than 50 words.
3. The questions can be asked under many conditions.
4. Make sure the answer is more than 10 words.
5. Make sure the answer can be obtained from the
    information provided.
6. Do not generate the same or similar answers as
    generated before.
7. There are three fields for your generation: {"
    instruction": <question>, "Input":"", "output": <
    answer> }.

Now, please generate the data in JSON format following
    the above requirements."
```

Listing 2: Instruction-Answer Generation Prompt

3.3 Training Pipeline

There are two stages in the training pipeline: pretraining and supervised fine-tuning.

3.4 Pretraining

When training LLM for HPC domain, the base model provides foundational language understanding, enabling quicker adaptation to the new domain's nuances. It offers transferable linguistic knowledge and reduces data requirements. The base model's broad comprehension aids in fine-tuning while retaining the ability to generate accurate responses beyond the specific domain. We select LLaMA [44] and LLaMA 2 [44] as base models for pretraining. The reason is two-fold. Firstly, the LLaMA series are open-source and free to conduct research, in contrast to the commercial nature of the GPT series. Secondly, LLaMA outperforms other models, like GPT-3, in efficiency and resource utilization. LLaMA series is a collection of multi-lingual base models with parameters ranging from 7 billion to 70 billion. Here, We adopt the 13B version for both LLaMA and LLaMA 2 models, ensuring training accessibility and achieving superior performance.

3.5 Supervised Fine-tuning

Although LLMs exhibit remarkable performance in general domains, their lack of domain-specific knowledge results in suboptimal performance in HPC fields that require specialized expertise. The HPC field's inherent nature necessitates models to possess comprehensive knowledge bases for relevant queries. Supervised fine-tuning has proven to be effective in tuning LLM for different tasks [40, 47]. Supervised fine-tuning helps the models perform satisfactorily under zero-shot scenarios with the cost of sufficient annotated instructions. Inspired by the automatic construction of the instruction along with the instances (inputs and outputs) [46], we generate our instruction data based on the above HPC knowledge with the proposed method in Section 3.2. Table 1 shows the supervised fine-tuning data examples for 2 HPC tasks. The data instance has an instruction that describes the tasks in natural language. In HPC-based tasks, we consider the instructions and input are the same and leave the input null value.

4 EXPERIMENT

4.1 Experiments Setting

Our experimental setup takes place on a DGX A100 node within the ThetaGPU server, equipped with eight NVIDIA A100 Tensor Core GPUs and two AMD Rome CPUs, providing 320 GB GPU memory. For training HPC-GPT, we allocate 12 hours with 200 training epochs and set the learning rate to 2e-5. To optimize the process, we utilize a training and evaluation batch size of 16. To efficiently fine-tune HPC-GPT, we leverage fp16 precision to reduce memory requirements. Furthermore, we apply the LoRA [27] technique, known as low-rank adaptation, to reduce the number of trainable parameters. Additionally, we employ the PEFT [33] technique, which stands for parameter-efficient fine-tuning, to further enhance the performance of the pre-trained models.

4.2 HPC Unstructured and Structured Data

We collect the HPC raw data from GitHub, papers, and websites. For the PLP task, the unstructured data is from more than 40 papers related to PLP tasks, such as paper [23], and the structured data is from tables in the paper (e.g., [38]) and GitHub (e.g., [5]). For the MLPerf task, the unstructured data is from papers related to ML performance such as paper [42], and the structured data is from tables such as the results in the website [2]. We feed the raw data to LLM using the methods in Section 3.2 to collect the instruction data.

4.3 Two HPC Tasks

1. Managing AI Models and Datasets has two subtasks: 1) PLP task and 2) MLPerf task. PLP task refers to leveraging machine learning techniques to understand and analyze programming languages in a human-readable manner. A large number of different ML models and datasets are published each year related to different kinds of PLP tasks such as code generation [32], clone detection [32], source-to-source translation [30], defect correction [9], code documentation [28] and so on. It is difficult for people, especially newcomers, to identify representative ones to get started. Different model architectures are used to solve different types of PLP tasks, ranging from program understanding to code generation. It is a daunting job for people to pick the right architectures for a given task. The goal of this task is to facilitate the reuse of datasets and models related to PLP tasks, so researchers or developers can easily create customized ML pipelines to solve a given task. MLPerf [42] is a standardized benchmark designed to evaluate and compare the inference performance of machine learning models and frameworks. The benchmark includes different types of models and uses cases. Participants in the MLPerf benchmark must run their inference engines on a predefined set of models and datasets, following specified guidelines for hardware and software configurations. The goal of MLPerf task is to help the participants know the current exact system (e.g., 16-nodes-SPR-pytorch), processor (e.g., Intel(R) Xeon(R) Platinum 8462Y+), accelerator (e.g, NVIDIA H100-SXM5-80GB), and software (e.g., PyTorch NVIDIA Release 23.04) that can facilitate them to build similar ML models and datasets efficiently.

2. Data Race Detection is a critical task in HPC. It is aimed at finding data race bugs in multithreaded programs such as those using OpenMP. In general, a data race occurs when two or more threads perform conflicting accesses (with at least one access being a write) to a shared variable without any synchronization among the threads. In this task, we cover two programming languages (such as C/C++ and Fortran) using OpenMP. The goal of this task is to detect if there is a data race problem given the code snippet.

4.4 Baselines

HPC-GPT comprises two versions: HPC-GPT (L1) and HPC-GPT (L2), each integrating the respective base models, LLaMa and LLaMa 2. In order to demonstrate the superior performance of HPC-GPT, we conduct a comparative analysis with different baseline methods.

Task 1: Managing AI Models and Datasets

- **ChatGPT** is the state of the art of commercial large language model, we use GPT-4 as its model engine with superior performance.
- **HPC Ontology [34]** is a unified ontology for managing AI models and datasets for HPC.

Task 2: Data Race Detection

- **LLOV [8]** is a fast, lightweight, language agnostic, and static data race checker for OpenMP programs based on the LLVM compiler framework.
- **Intel Inspector [29]** is a dynamic analysis tool that detects threading and memory errors in C, C++ and Fortran codes. It supersedes Intel's Thread Checker tool, with added memory error checking. Supported thread errors include race conditions and deadlocks.
- **ROMP [25]** is a tool for detecting data races in executions of scalable parallel applications that employ OpenMP for node-level parallelism.
- **Thread Sanitizer [43]** is a runtime data race detector developed by Google. ThreadSanitizer is now part of the LLVM and GCC compilers to enable data race detection for C++, Fortran and Go code.
- **GPT-3.5 and GPT-4** are standard commercial LLM developed by OpenAI. GPT-4 with 1.7 trillion parameters is the latest and most advanced version of GPT compared with GPT-3 with 175 billion parameters.

Table 1: Instance with An Instruction

Task1: Model and datasets for HPC
"instruction": "What kind of dataset can be used if the language is C/C++ and the baseline is CodeBERT?", "input":"", "output": "The POJ-104 dataset can be used for clone detection tasks if the language is C/C++ and the baseline is CodeBERT."
Task 2: Data Race Detection
"instruction": "Given the code snippet: "‘c\n#pragma omp parallel for\nfor (i = 1; i <n; i++)\n y[i] = x[i] + y[i - 1];\n"‘, help me detect if adding pragma will cause a data race problem? Answer 'yes' if it causes a data race problem and 'no' if it will not cause a data race problem.", "input":"", "output": "yes"

Table 2: Dataset Information for Task 1

Subtasks	Category	Number	Percentage
PLP	Performance Modeling	44	7.30%
	Algorithm Classification	41	6.80%
	Defect detection	47	7.79%
	Clone detection	45	7.46%
	Code Completion	39	6.47%
	Compiler Analyses	37	6.14%
	Code Repair	48	7.96%
	Code Translation	41	6.80%
	Cloze Testing	48	7.96%
	Text-to-Code Generation	58	9.62%
	Code Summarization	48	7.96%
	Document Translation	52	8.62%
	Code Search	55	9.12%
MLPerf	Submitter	324	17.80%
	System	386	21.21%
	Processor	347	19.07%
	Accelerator	362	19.89%
	Software	401	22.03%

- **LLaMA [44] and LLaMA 2 [45]** are standard open-source LLM developed by Meta. LLaMA 2 was trained on 40% more data than LLaMA. We use 13B versions for LLaMA and LLaMA 2 due to the tradeoff between performance and training costs.

4.5 Metrics

To evaluate HPC-GPT, we calculate counts of True Positive (TP), False Positive (FP), True Negative (TN), and False Negatives (FN) based on tool results and ground truth. We use five standard metrics: Recall, Specificity, Precision, Accuracy, and F1 score to evaluate the quality of tools. We also report tool support rate (TSR), which is the ratio of how many test files are supported by a tool. The F1 score is a measure combining both precision and recall. It provides a single metric that weights precision and recall in a balanced way, requiring both to have higher values for the F1-score value to rise. The reported adjusted F1 score, F1 score multiplied by the TSR, can show the true ability of a tool.

4.6 Instruction Datasets for HPC

We have collected a total of 5.86k instruction data for two HPC applications. The dataset details for task 1 and task 2 are presented in Table 2 and Table 3, respectively. In Table 2, we focus on task 1: Managing AI models and datasets for PLP tasks and MLPerf tasks. There are 13 categories in PLP tasks covering various programming language processing subtasks, such as clone detection and code repair. The percentage range for these categories is 6.14% to 9.62%. Additionally, there are 5 categories dedicated to MLPerf tasks, such as the system and software, with a percentage range of 17.80% to 22.03%. In Table 3, the data race detection task has two main categories: "code snippet with data races" and "code snippet without data races." We have summarized 7 common data race types for both C/C++ and Fortran, including "missing data sharing" and "Unresolvable dependencies." Additionally, we have identified 7 types of data without data races, such as "Single thread execution" and "Use of synchronization." The percentage range for these categories is 5.96% to 8.17% for C/C++ and 6.15% to 8.25% for Fortran.

Overall, we have ensured that the number of different categories is balanced for various subtasks. This approach prevents one category from dominating and allows the LLM to maintain a well-rounded knowledge base for each task. This diverse and comprehensive dataset collection is crucial in fine-tuning the LLM effectively and preparing it to tackle a wide array of HPC-specific challenges with improved accuracy and performance.

4.7 Preliminary Results

4.7.1 Managing AI models and Datasets. This section presents some use cases using HPC-GPT for providing the dataset and model information and answering questions.

1) PLP Task. The AI models and dataset are fundamental information for PLP tasks [11, 13, 14]. We use natural language to describe the code translation-related questions in [23].

```
Question: "What kind of dataset can be used for code
    translation tasks if the source language is Java and
     the target language is C#?"

Answer (GPT-4): "For code translation tasks from Java to
    C#, you would need a dataset that consists of pairs
    of Java code and their corresponding equivalent C#
    code. "

Answer (HPC-Ontology): "CodeTrans dataset"

Answer (HPC-GPT (L2)): "The CodeTrans dataset can be used
     for code translation tasks if the source language
    is Java and the target language is C#."
```

Listing 3: PLP Task Example

Table 3: Dataset Information for Task 2

Subtasks	Metric	Category													
		Code snippet with data races							Code snippet without data races						
		Unresolvable dependencies	Missing data sharing clauses	Missing synchronization	SIMD data races	Accelerator data races	Undefined behavior	Numerical kernel data races	Single thread execution	Use of data sharing clauses	Use of synchronization	Use of SIMD directives	Use of accelerator directives	Use of special language features	Numerical kernels
C/C++	Number	132	129	130	124	110	128	133	133	105	144	119	118	126	131
	Percentage	7.49%	7.32%	7.38%	7.04%	6.24%	7.26%	7.55%	7.55%	5.96%	8.17%	6.75%	6.70%	7.15%	7.43%
Fortran	Number	125	103	117	122	101	109	111	98	126	105	130	97	108	124
	Percentage	7.93%	6.54%	7.42%	7.74%	6.41%	6.91%	7.04%	6.21%	8.00%	6.66%	8.25%	6.15%	6.85%	7.86%

Table 4: Data Race Detection Tool and Compiler Version

Tools	Version	Compiler
ThreadSanitizer	10.0.0	Clang/LLVM 10.0.0
Intel Inspector	2021.1	Intel Compiler 2021.3.0
ROMP	20ac93c	GCC/gfortran 7.4.0
LLOV	N/A	Clang/LLVM 6.0.1

2) MLPerf Task. The system, processor, and software-related details are important for MLPerf tasks. We use natural language to describe the MLPerf-related questions in [2].

```
Question: "What is the System if the Accelerator used is
    NVIDIA H100-SXM5-80GB and the Software used is MXNet
     NVIDIA Release 23.04?"

Answer (GPT-4): "As of my last update in September 2021,
    the NVIDIA H100-SXM5-80GB is a data center GPU
    designed for high-performance computing and deep
    learning workloads. "

Answer (HPC-Ontology): "dgxh100_n64"

Answer (HPC-GPT (L2)): "If the Accelerator used is NVIDIA
     H100-SXM5-80GB and the Software used is MXNet
    NVIDIA Release 23.04, the System is dgxh100_n64."
```

Listing 4: MLPerf Task Example

Results. The results for the PLP task and MLPerf task are shown in Listing 3 and 4 respectively. Upon analyzing the outputs, it is evident that HPC-GPT demonstrates its superiority in handling these tasks compared to GPT-4. In the PLP task, when the user asks, "What kind of dataset can be used for code translation tasks if the source language is Java and the target language is C#?", HPC-GPT is capable of providing an exact answer with the "CodeTrans dataset," while GPT-4 merely repeats the question due to its lack of relevant knowledge. Similarly, in the MLPerf task, when the user presents a system-related question such as, "What is the System if the Accelerator used is NVIDIA H100 - SXM5 - 80 GB and the Software used is MXNet NVIDIA Release 23.04?", HPC-GPT promptly offers a specific answer, "dgxh100_n64," demonstrating its understanding of the question. Conversely, ChatGPT's response includes a general introduction of the Accelerator but fails to provide the correct answer, indicating its lack of relevant knowledge regarding the MLPerf task.

To address this limitation, HPC-Ontology can also deliver accurate answers by leveraging SPARQL query language. However, it requires manual effort to write SPARQL queries for different questions and answers, making it less scalable. HPC-GPT overcomes this challenge by effectively translating human language into embeddings, enabling it to process all the information and provide relevant answers to various queries. In both tasks, HPC-GPT demonstrates its versatility in handling different conditions and benefits users by allowing them to express their questions freely while receiving accurate and related answers.

4.7.2 Data Race Detection. In this section, we present the evaluation results of HPC-GPT on DataRaceBench V1.4.0 [15, 35] with 177 C/C++ test programs and 166 Fortran test programs. Among these, 88 C/C++ and 84 Fortran test cases exhibit data races, while 89 C/C++ and 82 Fortran test cases are free from data races. We utilize the metrics defined in Section 4.5 to assess the performance of data race detection tools (LLOV, Intel Inspector, ROMP, and Thread Sanitizer) and LLM-based methods (GPT-3.5, GPT-4, LLaMa, LLaMa2, HPC-GPT (L1) and HPC-GPT (L2)). The compiler version is shown in Table 4. The results are summarized in Table 5, where the best result for each metric is highlighted in bold. In C/C++ language, Thread Sanitizer excels in Adjusted F1 (0.8679), specificity (0.9888), precision (0.9857), and accuracy (0.8826). Remarkably, HPC-GPT (L2) secures second place in accuracy (0.8037), Adjusted F1 (0.8072), and Recall (0.8171). For Fortran, Thread Sanitizer shines with the best results in specificity (1.0), precision (1.0), and accuracy (0.8863). HPC-GPT (L2) leads in Recall (0.8433) and Adjusted F1 (0.8333).

Compared with LLM-based methods, both HPC-GPT (L1) and HPC-GPT (L2) demonstrate notable improvements over LLaMa, LLaMa 2, GPT-3.5, GPT-4 across five key metrics (recall, specificity, precision, accuracy, and adjusted F1). In C/C++ language, HPC-GPT (L2) achieves improvements of 36.11%, 34.84%, 26.33%, 11.1%, and 3.85% compared with LLaMa, LLaMa 2, GPT-3.5, GPT-4, and HPC-GPT (L1). For Fortran, HPC-GPT (L2) attains enhancements

Table 5: Results of Individual Data Race Detection Tools and LLM-Based methods.

Tool	Language	TP	FP	TN	FN	Recall	Specificity	Precision	Accuracy	TSR	Adjusted F1
LLOV	C/C++	58	**9**	78	29	0.6666	0.8965	0.8656	0.7816	0.9613	0.7532
Intel Inspector		**76**	41	46	**10**	**0.837**	0.5287	0.6495	0.7052	0.9558	0.7487
ROMP		63	12	65	18	0.775	0.8333	0.8266	0.8037	0.8729	0.8000
Thread Sanitizer		69	**1**	**89**	20	0.7752	**0.9888**	**0.9857**	**0.8826**	**0.9889**	**0.8679**
GPT-3.5		52	36	45	30	0.6341	0.5555	0.5909	0.5951	0.9209	0.6117
GPT-4		65	31	50	17	0.7926	0.6172	0.6770	0.7055	0.9209	0.73033
LLaMa		65	61	20	17	0.7926	0.2469	0.5158	0.52147	0.9209	0.625
LLaMa2		71	66	15	11	0.8658	0.1851	0.51824	0.5276	0.9209	0.6484
HPC-GPT (L1)		64	20	61	18	0.7804	0.7530	0.7619	0.7668	0.9209	0.7710
HPC-GPT (L2)		67	17	64	15	0.8171	0.7901	0.7976	0.8037	0.9209	0.8072
LLOV	Fortran	40	11	70	36	0.5263	0.8641	0.7843	0.7006	0.9457	0.6299
Intel Inspector		66	11	65	17	0.7951	0.8552	0.8571	0.8238	0.9464	0.825
ROMP		57	10	54	20	0.7402	0.8437	0.8507	0.7872	0.8392	0.7916
Thread Sanitizer		52	**0**	65	15	0.7761	**1.0**	**1.0**	**0.8863**	0.7857	0.8739
GPT-3.5		54	31	50	31	0.6352	0.6172	0.6352	0.6265	**1.0**	0.6352
GPT-4		67	37	44	18	0.7882	0.5432	0.6442	0.6687	**1.0**	0.7089
LLaMa		63	55	26	22	0.7411	0.3209	0.5338	0.5361	**1.0**	0.6206
LLaMa2		59	63	18	26	0.6941	0.2222	0.4836	0.4638	**1.0**	0.5700
HPC-GPT (L1)		66	20	61	19	0.7764	0.7530	0.7674	0.7650	**1.0**	0.7719
HPC-GPT (L2)		**70**	15	**68**	**13**	**0.8433**	0.8192	0.8235	0.8313	**1.0**	**0.8333**

of 31.89%, 35.23%, 21.34%, 15.79%, and 7.28% over LLaMa, LLaMa 2, GPT-3.5, GPT-4, and HPC-GPT (L1). Notably, all LLM-based methods share the same TSR due to an 8k token constraint, limiting input length. For C/C++, TSR is lower than existing tools, with 14 test cases exceeding 8k tokens. Conversely, Fortran's TSR for LLM-based methods is 1.0, surpassing existing tools.

5 CHALLENGES AND POSSIBLE SOLUTIONS

The Token Length of Existing LLMs: The limitation of LLM-based methods in detecting data races that exceed the 8k token limit poses a significant challenge in practical applications. When dealing with real-world software projects, data races may involve extensive sections of code that surpass the token limit. As a consequence, LLM-based methods fail to process such data race instances, leading to incomplete and potentially inaccurate results. To address this limitation, One approach is to investigate ways to extend the token limit of LLM models, allowing them to handle longer code snippets effectively. This involves optimizing the model architecture or leveraging advanced hardware configurations. Another avenue for improvement involves devising a pre-processing or partitioning mechanism to break down large code snippets into smaller, manageable segments that fit within the token limit. This way, LLM-based methods can analyze each segment individually and then combine the results to detect data races across the entire codebase.

How to update HPC-GPT with Latest Data: Updating HPC-GPT with the latest data poses a challenge due to the continuous release of datasets and models. Several strategies can be employed for effective updates. Initially, new data can be periodically gathered to retrain the entire HPC-GPT alongside existing data. Another method involves creating a checkpoint of the current model version and then resuming training using the newly acquired data. Another approach leverages the LangChain framework [1], wherein HPC-GPT integrates new data seamlessly. The Longchain APIs enable the storage of text within semantic vector stores. This integration process entails the division of text into chunks, followed by embedding and matching prompts with the most relevant vector chunks. Consequently, this enhances the context of responses while adhering to token limitations.

6 CONCLUSION

This paper proposes HPC-GPT, a large language model specifically designed for the HPC domain. We explore the capabilities of this model in addressing two common tasks in HPC and demonstrate its excellent performance. In the first task, concerning model and dataset selection for HPC, HPC-GPT exhibits the ability to retrieve and extract HPC-related datasets and models based on human expressions. It efficiently handles multiple conditions, providing valuable support for researchers in the HPC field. Additionally, we evaluate HPC-GPT's performance in data-race detection in an OpenMP program, a critical concern in parallel computing. Our experiments demonstrate that HPC-GPT achieves good performance compared to existing data-race detectors.

ACKNOWLEDGMENTS

This research was funded in part by and used resources at the Argonne Leadership Computing Facility, which is a DOE Office of Science User Facility supported under Contract DE-AC02-06CH11357. This work is also prepared by LLNL under Contract DE-AC52-07NA27344 (LLNL-CONF-853156) and supported by the U.S. Department of Energy, Office of Science, Advanced Scientific Computing Research.

REFERENCES

[1] 2023. LangChain: Next Generation Language Processing. https://langchain.com/ Accessed: 2023-05-15.

[2] 2023. MLPerf Training v3.0 Results. https://docs.google.com/spreadsheets/d/1bF4buOnEPQcwoqlaSeX4HxKx8jVRR0xHcOT_CaAL5Mk/pubhtml?gid=0&single=false&widget=false&headers=false&chrome=true

[3] 2023. OpenAI. ChatGPT. https://openai.com/blog/chatgpt/ Accessed: 2023-02-08.

[4] 2023. OpenAI Codex. https://openai.com/blog/openai-codex Accessed: 2023-03.

[5] 2023. Structed Data of PLP tasks. https://github.com/microsoft/CodeXGLUE Accessed: 2023-06-15.

[6] Zhiyu An, Xianzhong Ding, Arya Rathee, and Wan Du. 2023. CLUE: Safe Model-Based RL HVAC Control Using Epistemic Uncertainty Estimation. In *ACM BuildSys*.

[7] Stephen H Bach, Victor Sanh, Zheng-Xin Yong, Albert Webson, Colin Raffel, Nihal V Nayak, Abheesht Sharma, Taewoon Kim, M Saiful Bari, Thibault Fevry, et al. 2022. Promptsource: An integrated development environment and repository for natural language prompts. *arXiv preprint arXiv:2202.01279* (2022).

[8] Utpal Bora, Santanu Das, Pankaj Kukreja, Saurabh Joshi, Ramakrishna Upadrasta, and Sanjay Rajopadhye. 2020. Llov: A fast static data-race checker for openmp programs. *ACM Transactions on Architecture and Code Optimization (TACO)* 17, 4 (2020), 1–26.

[9] Jakob Božič, Domen Tabernik, and Danijel Skočaj. 2021. Mixed supervision for surface-defect detection: From weakly to fully supervised learning. *Computers in Industry* 129 (2021), 103459.

[10] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. 2023. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712* (2023).

[11] Le Chen, Xianzhong Ding, Murali Emani, Tristan Vanderbruggen, Pei-hung Lin, and Chuanhua Liao. 2023. Data Race Detection Using Large Language Models. *arXiv preprint arXiv:2308.07505* (2023).

[12] Le Chen, Pei-Hung Lin, Tristan Vanderbruggen, Chunhua Liao, Murali Emani, and Bronis de Supinski. 2023. LM4HPC: Towards Effective Language Model Application in High-Performance Computing. In *International Workshop on OpenMP*. Springer, 18–33.

[13] Le Chen, Quazi Ishtiaque Mahmud, and Ali Jannesari. 2022. Multi-view learning for parallelism discovery of sequential programs. In *2022 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 295–303.

[14] Le Chen, Quazi Ishtiaque Mahmud, Hung Phan, Nesreen Ahmed, and Ali Jannesari. 2023. Learning to Parallelize with OpenMP by Augmented Heterogeneous AST Representation. *Proceedings of Machine Learning and Systems* 5 (2023).

[15] Le Chen, Wenhao Wu, Stephen F Siegel, Pei-Hung Lin, and Chunhua Liao. 2023. DataRaceBench V1. 4.1 and DataRaceBench-ML V0. 1: Benchmark Suites for Data Race Detection. *arXiv preprint arXiv:2308.08473* (2023).

[16] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[17] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2022. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311* (2022).

[18] Xianzhong Ding, Le Chen, Murali Emani, Chunhua Liao, Pei-Hung Lin, Tristan Vanderbruggen, Zhen Xie, Alberto E. Cerpa, and Wan Du. 2023. HPC-GPT: Integrating Large Language Model for High-Performance Computing. In *Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis (SC-W 2023)*.

[19] Xianzhong Ding and Wan Du. 2022. Drlic: Deep reinforcement learning for irrigation control. In *2022 21st ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN)*. IEEE, 41–53.

[20] Xianzhong Ding and Wan Du. 2022. Smart irrigation control using deep reinforcement learning. In *2022 21st ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN)*. IEEE, 539–540.

[21] Xianzhong Ding, Wan Du, and Alberto Cerpa. 2019. OCTOPUS: Deep reinforcement learning for holistic smart building control. In *Proceedings of the 6th ACM international conference on systems for energy-efficient buildings, cities, and transportation*. 326–335.

[22] Xianzhong Ding, Wan Du, and Alberto E Cerpa. 2020. Mb2c: Model-based deep reinforcement learning for multi-zone building control. In *Proceedings of the 7th ACM international conference on systems for energy-efficient buildings, cities, and transportation*. 50–59.

[23] Patrick Flynn, Tristan Vanderbruggen, Chunhua Liao, Pei-Hung Lin, Murali Emani, and Xipeng Shen. 2022. Finding reusable machine learning components to build programming language processing pipelines. *arXiv preprint arXiv:2208.05596* (2022).

[24] William F Godoy, Pedro Valero-Lara, Keita Teranishi, Prasanna Balaprakash, and Jeffrey S Vetter. 2023. Evaluation of OpenAI Codex for HPC Parallel Programming Models Kernel Generation. *arXiv preprint arXiv:2306.15121* (2023).

[25] Yizi Gu and John Mellor-Crummey. 2018. Dynamic data race detection for OpenMP programs. In *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 767–778.

[26] Jeremy Howard and Sebastian Ruder. 2018. Universal language model fine-tuning for text classification. *arXiv preprint arXiv:1801.06146* (2018).

[27] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685* (2021).

[28] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. Codesearchnet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436* (2019).

[29] Intel Inspector. 2017. Intel Inspector 2017. https://software.intel.com/en-us/intel-inspector-xe.

[30] Marie-Anne Lachaux, Baptiste Roziere, Lowik Chanussot, and Guillaume Lample. 2020. Unsupervised translation of programming languages. *arXiv preprint arXiv:2006.03511* (2020).

[31] Guillaume Lample and Alexis Conneau. 2019. Cross-lingual language model pretraining. *arXiv preprint arXiv:1901.07291* (2019).

[32] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.

[33] Baohao Liao, Yan Meng, and Christof Monz. 2023. Parameter-Efficient Fine-Tuning without Introducing New Latency. *arXiv preprint arXiv:2305.16742* (2023).

[34] Chunhua Liao, Pei-Hung Lin, Gaurav Verma, Tristan Vanderbruggen, Murali Emani, Zifan Nan, and Xipeng Shen. 2021. Hpc ontology: Towards a unified ontology for managing training datasets and ai models for high-performance computing. In *2021 IEEE/ACM Workshop on Machine Learning in High Performance Computing Environments (MLHPC)*. IEEE, 69–80.

[35] Pei-Hung Lin and Chunhua Liao. 2021. High-precision evaluation of both static and dynamic tools using dataracebench. In *2021 IEEE/ACM 5th International Workshop on Software Correctness for HPC Applications (Correctness)*. IEEE, 1–8.

[36] Jie Liu, Bogdan Nicolae, and Dong Li. 2022. Lobster: Load Balance-Aware I/O for Distributed DNN Training. In *Proceedings of the 51st International Conference on Parallel Processing*. 1–11.

[37] Jie Liu, Bogdan Nicolae, Dong Li, Justin M Wozniak, Tekin Bicer, Zhengchun Liu, and Ian Foster. 2022. Large Scale Caching and Streaming of Training Data for Online Deep Learning. In *Proceedings of the 12th Workshop on AI and Scientific Computing at Scale using Flexible Computing Infrastructures*. 19–26.

[38] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. Codexglue: A machine learning benchmark dataset for code understanding and generation. *arXiv preprint arXiv:2102.04664* (2021).

[39] Boris Motik, Peter F Patel-Schneider, Bijan Parsia, Conrad Bock, Achille Fokoue, Peter Haase, Rinke Hoekstra, Ian Horrocks, Alan Ruttenberg, Uli Sattler, et al. 2009. OWL 2 web ontology language: Structural specification and functional-style syntax. *W3C recommendation* 27, 65 (2009), 159.

[40] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems* 35 (2022), 27730–27744.

[41] Hamid Rajabi, Xianzhong Ding, Wan Du, and Alberto Cerpa. 2023. TODOS: Thermal sensOr Data-driven Occupancy Estimation System for Smart Buildings. In *Proceedings of the 10th ACM international conference on systems for energy-efficient buildings, cities, and transportation*.

[42] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, et al. 2020. Mlperf inference benchmark. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 446–459.

[43] Konstantin Serebryany and Timur Iskhodzhanov. 2009. ThreadSanitizer: data race detection in practice. In *Proceedings of the workshop on binary instrumentation and applications*. 62–71.

[44] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).

[45] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288* (2023).

[46] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2022. Self-instruct: Aligning language model with self generated instructions. *arXiv preprint arXiv:2212.10560* (2022).

[47] Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. 2021. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652* (2021).

[48] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.

[49] Yiben Yang, Chaitanya Malaviya, Jared Fernandez, Swabha Swayamdipta, Ronan Le Bras, Ji-Ping Wang, Chandra Bhagavatula, Yejin Choi, and Doug Downey. 2020. Generative data augmentation for commonsense reasoning. *arXiv preprint arXiv:2004.11546* (2020).